

Informe Triggers Módulo 5

Gestión de Suministros

Versión: 1.0

5 de mayo de 2026

Juan David Peña Roldán

Universidad Surcolombiana

Neiva, Huila

2026

TGR-M05-01:

trg_inmutabilidad_registro_consumo_validado

Descripción	Impide cualquier operación UPDATE sobre un registro de consumo de alimento que ya se encuentre en estado VALIDADO. Se dispara en BEFORE UPDATE sobre registros_consumo_alimentos. Si el registro en su estado actual (OLD.estado_registro) es 'VALIDADO' y el campo que se intenta cambiar no es el campo de estado (es decir, no es una transición a 'ANULADO'), lanza una excepción. Permite únicamente la transición VALIDADO → ANULADO.
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Restricciones, punto 5:</i> "Un registro en estado VALIDADO es inmutable. No puede ser editado. Solo puede ser ANULADO por el Veterinario o Administrador con campo justificacion_anulacion obligatorio (mínimo 50 caracteres)." <i>RF-75, Flujo alterno E5:</i> "El usuario intenta modificar un registro con estado VALIDADO. HTTP 405: Method Not Allowed."
Justificación	La inmutabilidad de un registro validado es una garantía de integridad del dato, no una regla de negocio de presentación. Si el backend fallara o se hiciera un acceso directo a la BD, esta restricción debe sostenerse. Es la última línea de defensa.

```
CREATE OR REPLACE FUNCTION
modulo5.fn_inmutabilidad_registro_consumo_validado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
```

```

-- Solo bloquea si el registro estaba en VALIDADO
-- Permite únicamente la transición de estado VALIDADO →
ANULADO
    IF OLD.estado_registro = 'VALIDADO' THEN
        -- Si se intenta cambiar cualquier campo que no sea el
        estado
        -- hacia ANULADO, se rechaza
        IF NEW.estado_registro = 'VALIDADO' THEN
            RAISE EXCEPTION
                'INMUTABILIDAD_VIOLADA: El registro de consumo con
id % está en estado VALIDADO y no puede ser modificado. Solo se
permite la transición a ANULADO.',
                OLD.id_consumo_alimeto
            USING ERRCODE = 'P0001';
        END IF;
        -- Si el nuevo estado es ANULADO, solo permite que cambien
        -- los campos propios de anulación; el resto debe
mantenerse
        IF NEW.estado_registro = 'ANULADO' THEN
            -- Protege campos de negocio contra modificación
            durante la anulación
            IF NEW.id_activo_biologico IS DISTINCT FROM
OLD.id_activo_biologico OR
                NEW.id_tipo_alimento IS DISTINCT FROM
OLD.id_tipo_alimento OR
                NEW.tipo_alimento IS DISTINCT FROM
OLD.tipo_alimento OR
                NEW.cantidad_suministrada IS DISTINCT FROM
OLD.cantidad_suministrada OR
                NEW.costo_total IS DISTINCT FROM
OLD.costo_total OR
                NEW.fecha_inicio_periodo IS DISTINCT FROM
OLD.fecha_inicio_periodo OR
                NEW.fecha_fin_periodo IS DISTINCT FROM
OLD.fecha_fin_periodo
            THEN
                RAISE EXCEPTION
                    'INMUTABILIDAD_VIOLADA: Durante la anulación
del registro %, no se pueden modificar campos de negocio. Solo se
permite cambiar el estado y los campos de anulación.',
                    OLD.id_consumo_alimeto
                USING ERRCODE = 'P0001';
            END IF;
        END IF;
    END IF;

```

```

        END IF;
    END IF;
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER trg_inmutabilidad_registro_consumo_validado
    BEFORE UPDATE ON modulo5.registros_consumo_alimentos
    FOR EACH ROW
    EXECUTE FUNCTION
modulo5.fn_inmutabilidad_registro_consumo_validado();

```

TGR-M05-02:

trg_inmutabilidad_registro_medicamento_validado

Descripción	Impide cualquier operación UPDATE sobre un registro de aplicación de medicamento que ya se encuentre en estado VALIDADO, permitiendo únicamente la transición a ANULADO. Se dispara en BEFORE UPDATE sobre registros_medicamentos. Si el estado actual es 'VALIDADO' y el nuevo estado también es 'VALIDADO' (intento de edición), lanza excepción. Si el estado actual ya es 'ANULADO' e intenta anularse de nuevo, lanza excepción. Protege campos de negocio durante la transición de anulación.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	<i>RF-76, Restricciones, punto 5:</i> "Un registro de aplicación en estado VALIDADO es inmutable. Solo puede ser anulado por el Veterinario o Administrador con justificacion_anulacion obligatoria (mínimo 20 caracteres)." <i>RF-76, Flujo alterno E8:</i> "El veterinario o administrador intenta

	anular un registro cuyo estado ya es ANULADO. HTTP 409: Conflict."
Justificación	Misma razón que TGR-M05-01. La inmutabilidad del registro médico es crítica para la trazabilidad sanitaria y la integridad del historial clínico del activo.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_inmutabilidad_registro_medimento_validado()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Bloquear re-anulación de un registro ya ANULADO
    IF OLD.estado_registro = 'ANULADO' THEN
        RAISE EXCEPTION
            'RE_ANULACION_INVALIDA: El registro de medicamento con id %
            ya se encuentra en estado ANULADO. No es posible anular un registro ya
            anulado.',
            OLD.id_registro_medimento
            USING ERRCODE = 'P0001';
    END IF;

    -- Bloquear edición de campos en registro VALIDADO
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'VALIDADO' THEN
        RAISE EXCEPTION
            'INMUTABILIDAD_VIOLADA: El registro de medicamento con id %
            está en estado VALIDADO y no puede ser modificado. Solo se permite la
            transición a ANULADO.',
            OLD.id_registro_medimento
            USING ERRCODE = 'P0001';
    END IF;

    -- Durante transición VALIDADO → ANULADO, proteger campos de negocio
    IF OLD.estado_registro = 'VALIDADO' AND NEW.estado_registro =
'ANULADO' THEN
        IF NEW.id_activo_biologico      IS DISTINCT FROM
OLD.id_activo_biologico      OR
        NEW.nombre_medimento          IS DISTINCT FROM
OLD.nombre_medimento          OR
        NEW.cantidad                  IS DISTINCT FROM OLD.cantidad
OR
        NEW.fecha_aplicacion          IS DISTINCT FROM
OLD.fecha_aplicacion          OR

```

```

        NEW.costo_unitario_medicamento IS DISTINCT FROM
OLD.costo_unitario_medicamento OR
        NEW.costo_total_medicamento IS DISTINCT FROM
OLD.costo_total_medicamento
    THEN
        RAISE EXCEPTION
            'INMUTABILIDAD_VIOLADA: Durante la anulación del
registro de medicamento %, no se pueden modificar campos de negocio.',
            OLD.id_registro_medicamento
        USING ERRCODE = 'P0001';

    END IF;
END IF;

RETURN NEW;
END;
$$;

CREATE TRIGGER trg_inmutabilidad_registro_medicamento_validado
    BEFORE UPDATE ON modulo5.registros_medicamentos
    FOR EACH ROW
    EXECUTE FUNCTION
modulo5.fn_inmutabilidad_registro_medicamento_validado();

```

TGR-M05-03:

trg_calculo_costo_total_consumo_alimento

Descripción	
	Calcula automáticamente el campo costo_total en registros_consumo_alimentos al momento del INSERT, tomando como base cantidad_suministrada y el costo_unitario del tipo de alimento referenciado. Se dispara en BEFORE INSERT sobre registros_consumo_alimentos. Obtiene el costo_unitario del tipo de alimento desde modulo5.tipos_alimentos y calcula costo_total = cantidad_suministrada × costo_unitario. Si ya viene calculado desde el backend, lo sobrescribe con el valor correcto (fuente de verdad). Valida que

	<code>cantidad_suministrada > 0.</code>
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Proceso, paso 8.1:</i> "El sistema calcula: <code>costo_total_alimento = cantidad_alimento × costo_unitario</code> " <i>RF-75, Salida, punto 3:</i> "Costo total de la alimentación calculado y persistido."
Justificación	El cálculo <code>costo_total = cantidad × costo_unitario</code> es puramente matemático, determinístico y depende de datos que ya están en la misma transacción. Garantiza consistencia incluso si el backend olvida calcularlo. Es un derivado simple que la BD puede garantizar sin lógica externa.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_calculo_costo_total_consumo_alimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_costo_unitario modulo5.tipos_alimentos.costo_unitario%TYPE;
BEGIN
    -- Validar cantidad positiva (refuerzo de constraint)
    IF NEW.cantidad_suministrada <= 0 THEN
        RAISE EXCEPTION
            'CANTIDAD_INVALIDA: La cantidad suministrada debe ser mayor
a cero. Valor recibido: %',
            NEW.cantidad_suministrada
            USING ERRCODE = 'P0001';
    END IF;

    -- Obtener costo unitario desde el catálogo de tipos de alimentos
    SELECT costo_unitario
    INTO v_costo_unitario
    FROM modulo5.tipos_alimentos
    WHERE id_tipo_elemento = NEW.id_tipo_alimento
        AND estado = 'ACTIVO';

    IF NOT FOUND THEN
        RAISE EXCEPTION

```

```

        'TIPO_ALIMENTO_INVALIDO: No se encontró el tipo de alimento
con id % en estado ACTIVO.',
        NEW.id_tipo_alimento
        USING ERRCODE = 'P0001';
    END IF;

    -- Calcular y asignar el costo total (fuente de verdad en BD)
    NEW.costo_total := NEW.cantidad_suministrada * v_costo_unitario;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_calculo_costo_total_consumo_alimento
    BEFORE INSERT ON modulo5.registros_consumo_alimentos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_calculo_costo_total_consumo_alimento();

```

TGR-M05-04:

trg_calculo_costo_total_med medicamento

Descripción	Calcula automáticamente el campo costo_total_med medicamento al momento del INSERT en registros_medicamentos, como producto de cantidad × costo_unitario_med medicamento. Se dispara en BEFORE INSERT sobre registros_medicamentos. Calcula costo_total_med medicamento = cantidad × costo_unitario_med medicamento. Valida que cantidad > 0 y que costo_unitario_med medicamento > 0.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	<i>RF-76, Proceso, paso 8a:</i> "El sistema calcula: costo_total_medicacion = costo_unitario × dosis_aplicada" <i>RF-76, Salida, punto 4:</i> "Costo total de la medicación calculado y persistido."
Justificación	Mismo principio que TGR-M05-03. Es un

	cálculo matemático simple y determinístico. Garantiza que el campo derivado sea siempre consistente con sus componentes, independientemente del canal de inserción.
--	--

```
CREATE OR REPLACE FUNCTION modulo5.fn_calculo_costo_total_medimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- Validar cantidad positiva
    IF NEW.cantidad <= 0 THEN
        RAISE EXCEPTION
            'CANTIDAD_INVALIDA: La dosis aplicada debe ser mayor a cero.
Valor recibido: %',
            NEW.cantidad
        USING ERRCODE = 'P0001';
    END IF;

    -- Validar costo unitario positivo
    IF NEW.costo_unitario_medimento <= 0 THEN
        RAISE EXCEPTION
            'COSTO_INVALIDO: El costo unitario del medicamento debe ser
mayor a cero. Valor recibido: %',
            NEW.costo_unitario_medimento
        USING ERRCODE = 'P0001';
    END IF;

    -- Calcular y asignar el costo total (fuente de verdad en BD)
    NEW.costo_total_medimento := NEW.cantidad *
NEW.costo_unitario_medimento;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_calculo_costo_total_medimento
BEFORE INSERT ON modulo5.registros_medamentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_calculo_costo_total_medimento();
```

TGR-M05-05:

trg_proteccion_delete_consumo_alimento

Descripción	Bloquea cualquier intento de eliminación física (DELETE) sobre registros de consumo de alimentos, independientemente de su estado. Se dispara en BEFORE DELETE sobre registros_consumo_alimentos para cada fila. Siempre lanza una excepción, bloqueando la operación sin excepción posible.
Aplicado a	RF-75 — Registro de Consumo de Alimentos / RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-75, Restricciones, punto 6:</i> "Un registro ANULADO no puede ser reactivado. Si se necesita reemplazarlo, se debe crear un nuevo registro." (implica que los registros nunca se eliminan) <i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-75, Postcondiciones, punto 4:</i> "Un registro ANULADO queda excluido de todos los cálculos pero permanece visible en el historial."
Justificación	La política append-only es una garantía de trazabilidad que debe imponerse a nivel de motor de base de datos. Ninguna lógica de aplicación, migración mal escrita o acceso directo debe poder borrar el historial de suministros.

```
CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_delete_consumo_alimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
```

```

BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de consumo de alimentos
        son append-only. No se permite la eliminación del registro %. Para
        invalidar un registro, utilice la operación de anulación.',
        OLD.id_consumo_alimento
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_consumo_alimento
    BEFORE DELETE ON modulo5.registros_consumo_alimentos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_delete_consumo_alimento();

```

TGR-M05-06: trg_proteccion_delete_medicamento

Descripción	Bloquea cualquier intento de eliminación física (DELETE) sobre registros de aplicación de medicamentos. Se dispara en BEFORE DELETE sobre registros_medicamentos. Siempre lanza una excepción.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos / RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION). No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-76, Restricciones, punto 5:</i> El historial de tratamientos debe conservarse íntegro para trazabilidad sanitaria.
Justificación	Misma razón que TGR-M05-05. La integridad del historial farmacológico es crítica para auditorías del ICA y trazabilidad NIC 41. No puede depender de restricciones

	de aplicación.
--	----------------

```
CREATE OR REPLACE FUNCTION modulo5.fn_proteccion_delete_medicamento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de aplicación de
        medicamentos son append-only. No se permite la eliminación del registro
        %. Para invalidar un registro, utilice la operación de anulación.',
        OLD.id_registro_medicamento
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_medicamento
    BEFORE DELETE ON modulo5.registros_medicamentos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_delete_medicamento();
```

TGR-M05-07:

trg_proteccion_delete_reporte_gastos

Descripción	Bloquea la eliminación física de reportes de gastos acumulados una vez generados. Se dispara en BEFORE DELETE sobre reporte_gastos_acumulados. Siempre lanza una excepción bloqueando el DELETE.
Aplicado a	RF-77 — Reporte de Gastos Acumulados
Restricción origen RF	<i>RF-77, Postcondiciones, punto 2:</i> "El documento exportado queda temporalmente disponible para descarga por el usuario durante 24 horas." (la expiración aplica a archivos, no al registro del reporte) <i>RF-77, Requerimientos no funcionales — Historial de reportes:</i> "Los parámetros guardados no tienen fecha de expiración y permanecen disponibles en el historial del usuario"

	indefinidamente."
Justificación	El registro del reporte en BD debe ser permanente para el historial del usuario. La expiración de archivos exportados es responsabilidad del backend/storage, no implica eliminar el registro de la tabla.

```
CREATE OR REPLACE FUNCTION modulo5.fn_proteccion_delete_reporte_gastos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los reportes de gastos acumulados no
        pueden eliminarse. El registro con id % debe conservarse en el historial
        de reportes del usuario de forma indefinida.',
        OLD.id_reporte_gasto_acumulado
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_reporte_gastos
    BEFORE DELETE ON modulo5.reporte_gastos_acumulados
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_proteccion_delete_reporte_gastos();
```

TGR-M05-08:

trg_proteccion_delete_costos_productivos

Descripción	Bloquea la eliminación de registros de costos productivos una vez consolidados, garantizando la inmutabilidad del acumulado del ciclo. Se dispara en BEFORE DELETE sobre costos_productivos. Siempre lanza una excepción.
Aplicado a	RF-78 — Acumulación de Inversión

Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only. Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION) con referencia al registro original (id_registro_original) y motivo_correccion obligatorio. No se permiten DELETE ni UPDATE sobre registros históricos." <i>RF-78, Restricciones, punto 2:</i> "Una vez cerrado el ciclo por RF-41, el monto_inversion_total_ciclo es inmutable. No se admiten nuevos registros ni modificaciones sobre ese ciclo."
Justificación	La inmutabilidad del acumulado de ciclo es un requisito NIC 41. Debe ser garantizada a nivel de motor.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_delete_costos_productivos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de costos productivos son
        append-only bajo política NIC 41. No se permite la eliminación del
        registro %. Las correcciones deben realizarse mediante registros de
        ajuste con tipo_operacion = CORRECCION.',
        OLD.id_costo_productivo
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_delete_costos_productivos
BEFORE DELETE ON modulo5.costos_productivos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_proteccion_delete_costos_productivos();

```

TGR-M05-09:

trg_validacion_fecha_consumo_no_futura

Descripción	Valida que la fecha de consumo de alimento no sea posterior a la fecha actual del servidor al momento del INSERT. Se dispara en BEFORE INSERT sobre registros_consumo_alimentos. Compara fecha_inicio_periodo (usada como fecha_consumo en el modelo actual) contra NOW(). Si es futura, lanza excepción.
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Restricciones, punto 2:</i> "La fecha_consumo no puede ser posterior a la fecha actual del sistema (no se permiten registros futuros)." <i>RF-75, Flujo alterno E2:</i> "fecha_consumo > fecha_actual del servidor. HTTP 400: Bad Request."
Justificación	Aunque el backend valida esto, la restricción de fecha no futura sobre datos históricos es una regla de integridad temporal que la BD puede y debe reforzar. Previene inserciones directas o migraciones con datos inválidos.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_validacion_fecha_consumo_no_futura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_inicio_periodo IS NOT NULL AND NEW.fecha_inicio_periodo
    > NOW() THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha de consumo (%) es
            posterior a la fecha actual del sistema (%). No se permiten registros de
            consumo con fechas futuras.',
            NEW.fecha_inicio_periodo,
            NOW()
            USING ERRCODE = 'P0001';
    END IF;
    RETURN NEW;
END;
$$;

```

```
CREATE TRIGGER trg_validacion_fecha_consumo_no_futura
  BEFORE INSERT ON modulo5.registros_consumo_alimentos
  FOR EACH ROW
  EXECUTE FUNCTION modulo5.fn_validacion_fecha_consumo_no_futura();
```

TGR-M05-10:

trg_validacion_fecha_medicamento_no_futura

Descripción	Valida que la fecha de aplicación del medicamento no sea posterior a la fecha actual del servidor. Se dispara en BEFORE INSERT sobre registros_medicamentos. Valida que fecha_aplicacion no sea mayor a CURRENT_DATE.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos
Restricción origen RF	<i>RF-76, Flujo alterno E2:</i> "fecha_aplicacion es posterior a la fecha actual del servidor. HTTP 400: Bad Request. Mensaje: 'La fecha de aplicación [FECHA_INGRESADA] es posterior a la fecha actual del sistema.'"
Justificación	Misma razón que TGR-M05-09. La BD es la última línea de defensa para integridad temporal de registros médicos.

```
CREATE OR REPLACE FUNCTION
modulo5.fn_validacion_fecha_medicamento_no_futura()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
  IF NEW.fecha_aplicacion > CURRENT_DATE THEN
    RAISE EXCEPTION
      'FECHA_FUTURA_NO_PERMITIDA: La fecha de aplicación del
medicamento (%) es posterior a la fecha actual del sistema (%). No se
```

```

permiten registros con fechas futuras.',
    NEW.fecha_aplicacion,
    CURRENT_DATE
    USING ERRCODE = 'P0001';
END IF;
RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validacion_fecha_medicamento_no_futura
    BEFORE INSERT ON modulo5.registros_medicamentos
    FOR EACH ROW
    EXECUTE FUNCTION
modulo5.fn_validacion_fecha_medicamento_no_futura();

```

TGR-M05-11:

trg_auditoria_consumo_alimento

Descripción	Registra automáticamente en auditorias_suministros todo INSERT o UPDATE (transición de estado) sobre registros_consumo_alimentos, capturando el estado anterior y posterior del registro. Se dispara en AFTER INSERT OR UPDATE sobre registros_consumo_alimentos. En INSERT registra la operación 'REGISTRO' con datos_nuevos. En UPDATE registra 'ANULACION' con datos_anteriores y datos_nuevos.
Aplicado a	RF-75 — Registro de Consumo de Alimentos / RF-80 — Auditoría y Trazabilidad
Restricción origen RF	<i>RF-75, Criterios de aceptación CA-7:</i> "El sistema registra en auditoría toda operación de registro, validación y anulación con usuario y timestamp." <i>RF-80, Descripción:</i> "El sistema debe registrar de forma automática, continua e inmutable todos los eventos relevantes generados por el módulo M05." <i>RF-75, Proceso, paso 12:</i> "El sistema registra la operación completa en el módulo

	de auditoría."
Justificación	La auditoría en BD garantiza que cada cambio quede registrado incluso si el backend no lo hace explícitamente. Es la capa de trazabilidad más confiable para registros que no pueden eliminarse y cuyo estado importa para NIC 41.

```

CREATE OR REPLACE FUNCTION modulo5.fn_auditoria_consumo_alimento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_tipo_op modulo5.enum_auditoria_suministro_tipo_operacion;
    v_datos_ant json := NULL;
    v_datos_nue json := NULL;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_tipo_op := 'REGISTRO';
        v_datos_nue := row_to_json(NEW);
    ELSIF TG_OP = 'UPDATE' THEN
        v_tipo_op := 'ANULACION';
        v_datos_ant := row_to_json(OLD);
        v_datos_nue := row_to_json(NEW);
    END IF;

    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        resultado
    ) VALUES (
        'registros_consumo_alimentos',
        v_tipo_op,
        v_datos_ant,
        v_datos_nue,
        COALESCE(NEW.id_usuario, OLD.id_usuario),
        NULL, -- IP gestionada por backend/sesión
        NOW(),
        'EXITOSO'
    );

```

```

        RETURN NULL; -- AFTER trigger, valor ignorado
END;
$$;

CREATE TRIGGER trg_auditoria_consumo_alimento
AFTER INSERT OR UPDATE ON modulo5.registros_consumo_alimentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_auditoria_consumo_alimento();

```

TGR-M05-12: trg_auditoria_medicamento

Descripción	Registra automáticamente en auditorias_suministros todo INSERT o UPDATE sobre registros_medicamentos. Se dispara en AFTER INSERT OR UPDATE sobre registros_medicamentos. Registra en auditorias_suministros la operación correspondiente.
Aplicado a	RF-76 — Registro de Aplicación de Medicamentos / RF-80 — Auditoría y Trazabilidad
Restricción origen RF	<i>RF-76, Criterios de aceptación CA-6:</i> "Toda operación de registro queda en auditoría con usuario, timestamp y detalle clínico." <i>RF-80, Catálogo de eventos M05:</i> "RF-78 produce: SUMINISTRO_REGISTRADO, SUMINISTRO_CORREGIDO..." <i>RF-76, Requerimientos no funcionales — Trazabilidad:</i> "El historial de tratamientos debe ser consultable desde la ficha del activo (RF-47) con todos los detalles."
Justificación	Mismo principio que TGR-M05-11. La trazabilidad clínica de tratamientos farmacológicos es un requisito regulatorio (ICA) que no puede quedar solo en manos del backend.

```

CREATE OR REPLACE FUNCTION modulo5.fn_auditoria_medicamento()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_tipo_op modulo5.enum_auditoria_suministro_tipo_operacion;
    v_datos_ant json := NULL;
    v_datos_nue json := NULL;
BEGIN
    IF TG_OP = 'INSERT' THEN
        v_tipo_op := 'REGISTRO';
        v_datos_nue := row_to_json(NEW);
    ELSIF TG_OP = 'UPDATE' THEN
        v_tipo_op := 'ANULACION';
        v_datos_ant := row_to_json(OLD);
        v_datos_nue := row_to_json(NEW);
    END IF;

    INSERT INTO modulo5.auditorias_suministros (
        entidad_afectada,
        tipo_operacion,
        datos_anteriores,
        datos_nuevos,
        id_usuario,
        ip_origen,
        fecha_evento,
        resultado
    ) VALUES (
        'registros_medicamentos',
        v_tipo_op,
        v_datos_ant,
        v_datos_nue,
        COALESCE(NEW.id_usuario, OLD.id_usuario),
        NULL,
        NOW(),
        'EXITOSO'
    );

    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_auditoria_medicamento
AFTER INSERT OR UPDATE ON modulo5.registros_medicamentos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_auditoria_medicamento();

```

TGR-M05-13:

trg_auditoria_costos_productivos

Descripción	Registra automáticamente en auditorias_suministros todo INSERT sobre costos_productivos, garantizando trazabilidad de cada acumulación de inversión. Se dispara en AFTER INSERT sobre costos_productivos. Inserta un registro en auditorias_suministros con tipo operación 'ACUMULACION' y los datos del nuevo registro.
Aplicado a	RF-78 — Acumulación de Inversión / RF-80 — Auditoría
Restricción origen RF	<i>RF-78, Postcondiciones, punto 4:</i> "La operación queda registrada en la bitácora de RF-80/M12." <i>RF-80, Catálogo de eventos:</i> "RF-78 produce: SUMINISTRO_REGISTRADO, ACUMULADO_ACTUALIZADO, CICLO_CONSOLIDADO." <i>RF-78, Criterios CA-7:</i> "El cálculo y su resultado quedan registrados en auditoría con usuario, fecha, id_activo y resultado."
Justificación	Cada inserción en costos productivos representa un evento contable NIC 41. Su registro en auditoría debe ser atómico con la inserción misma, no dependiente de una llamada posterior del backend.

```
CREATE OR REPLACE FUNCTION modulo5.fn_auditoria_costos_productivos()  
RETURNS TRIGGER  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    INSERT INTO modulo5.auditorias_suministros (  
        entidad_afectada,  
        tipo_operacion,  
        datos_anteriores,  
        datos_nuevos,
```

```

        id_usuario,
        ip_origen,
        fecha_evento,
        resultado
    ) VALUES (
        'costos_productivos',
        'ACUMULACION',
        NULL,
        row_to_json(NEW),
        NEW.id_usuario,
        NULL,
        NOW(),
        'EXITOSO'
    );

    RETURN NULL;
END;
$$;

CREATE TRIGGER trg_auditoria_costos_productivos
    AFTER INSERT ON modulo5.costos_productivos
    FOR EACH ROW
    EXECUTE FUNCTION modulo5.fn_auditoria_costos_productivos();

```

TGR-M05-14:

trg_proteccion_update_costos_productivos

Descripción	Bloquea cualquier UPDATE directo sobre registros de costos_productivos, forzando que las correcciones se realicen únicamente mediante nuevos registros de ajuste. Se dispara en BEFORE UPDATE sobre costos_productivos. Siempre lanza una excepción indicando que se debe usar un registro de corrección.
Aplicado a	RF-78 — Acumulación de Inversión
Restricción origen RF	<i>RF-78, Restricciones, punto 6:</i> "Los registros de suministros son append-only.

	Las correcciones generan un registro de ajuste (tipo_operacion = CORRECCION) con referencia al registro original (id_registro_original) y motivo_correccion obligatorio. No se permiten DELETE ni UPDATE sobre registros históricos." RF-78, Restricciones, punto 2: "Una vez cerrado el ciclo por RF-41, el monto_inversion_total_ciclo es inmutable."
Justificación	La política append-only del acumulado de inversión es un requisito NIC 41. Ningún UPDATE sobre registros históricos debe ser posible, sin excepción.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_proteccion_update_costos_productivos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    RAISE EXCEPTION
        'OPERACION_NO_PERMITIDA: Los registros de costos productivos son
inmutables (append-only, NIC 41). No se permite modificar el registro %.
Para realizar una corrección, inserte un nuevo registro con
tipo_operacion = CORRECCION referenciando el id del registro original.',
        OLD.id_costo_productivo
    USING ERRCODE = 'P0001';
END;
$$;

CREATE TRIGGER trg_proteccion_update_costos_productivos
BEFORE UPDATE ON modulo5.costos_productivos
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_proteccion_update_costos_productivos();

```

TGR-M05-15:

trg_validacion_estado_tipo_alimento_a
ctivo

Descripción	Valida al momento del INSERT en
--------------------	---------------------------------

	<code>registros_consumo_alimentos</code> que el tipo de alimento referenciado esté en estado ACTIVO en el catálogo. Se dispara en BEFORE INSERT sobre <code>registros_consumo_alimentos</code>. Verifica que el tipo de alimento esté en estado 'ACTIVO'. Si no, lanza excepción. (Nota: este trigger convive con TGR-M05-03 ya que este verifica el estado y el otro calcula el costo; ambos están en el mismo evento pero hacen cosas distintas. En la implementación final pueden fusionarse en una sola función o mantenerse separados por claridad de responsabilidades.)
Aplicado a	RF-75 — Registro de Consumo de Alimentos
Restricción origen RF	<i>RF-75, Precondiciones, punto 3:</i> "El activo biológico debe estar registrado en el sistema y en estado ACTIVO ." (por extensión, los elementos del catálogo también deben estar activos para ser usados) <i>RF-75, Proceso, paso 2:</i> "El sistema valida: El <code>activo_biologico_id</code> existe y está en estado ACTIVO ."
Justificación	La integridad referencial básica (FK) asegura que el <code>id_tipo_alimento</code> exista, pero no que esté ACTIVO . Este trigger complementa la FK para validar el estado lógico, no solo la existencia física del registro.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_validacion_estado_tipo_alimento_activo()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    v_estado modulo5.enum_tipo_alimento_estado;
BEGIN
    SELECT estado
    INTO v_estado
    FROM modulo5.tipos_alimentos
    WHERE id_tipo_elemento = NEW.id_tipo_alimento;

```

```

    IF NOT FOUND THEN
        RAISE EXCEPTION
            'TIPO_ALIMENTO_NO_ENCONTRADO: El tipo de alimento con id %
no existe en el catálogo.',
            NEW.id_tipo_alimento
            USING ERRCODE = 'P0001';
    END IF;

    IF v_estado != 'ACTIVO' THEN
        RAISE EXCEPTION
            'TIPO_ALIMENTO_INACTIVO: El tipo de alimento con id % está
en estado % y no puede usarse para nuevos registros de consumo.',
            NEW.id_tipo_alimento,
            v_estado
            USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validacion_estado_tipo_alimento_activo
    BEFORE INSERT ON modulo5.registros_consumo_alimentos
    FOR EACH ROW
    EXECUTE FUNCTION
modulo5.fn_validacion_estado_tipo_alimento_activo();

```

TGR-M05-16:

trg_consistencia_fechas_reporte_gastos

Descripción	
	Valida que en reporte_gastos_acumulados la fecha_inicio_reporte no sea posterior a fecha_fin_report y que fecha_fin_report no sea futura al momento de la inserción. Se dispara en BEFORE INSERT OR UPDATE sobre reporte_gastos_acumulados. Valida que fecha_inicio_reporte <= fecha_fin_report y que fecha_fin_report <= CURRENT_DATE.

Aplicado a	RF-77 — Reporte de Gastos Acumulados
Restricción origen RF	<i>RF-77, Restricciones, punto 2:</i> "El rango de fechas del reporte no puede ser futuro. $\text{fecha_fin_reporte} \leq \text{fecha_actual del sistema.}$ " <i>RF-77, Flujo alterno E2:</i> "Rango de fechas inválido (inicio > fin). HTTP 400: Bad Request. Mensaje: 'El rango de fechas es inválido. La fecha de inicio no puede ser posterior a la fecha de fin.'"
Justificación	Las restricciones de rango de fechas en tablas de reportes son integridad de datos que la BD puede garantizar sin ambigüedad, complementando la validación del backend.

```

CREATE OR REPLACE FUNCTION
modulo5.fn_consistencia_fechas_reporte_gastos()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.fecha_inicio_reporte > NEW.fecha_fin_reporte THEN
        RAISE EXCEPTION
            'RANGO_FECHAS_INVALIDO: La fecha de inicio del reporte (%)
no puede ser posterior a la fecha de fin (%).',
            NEW.fecha_inicio_reporte,
            NEW.fecha_fin_reporte
            USING ERRCODE = 'P0001';
    END IF;

    IF NEW.fecha_fin_reporte > CURRENT_DATE THEN
        RAISE EXCEPTION
            'FECHA_FUTURA_NO_PERMITIDA: La fecha de fin del reporte (%)
es posterior a la fecha actual del sistema (%). Los reportes no pueden
cubrir períodos futuros.',
            NEW.fecha_fin_reporte,
            CURRENT_DATE
            USING ERRCODE = 'P0001';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_consistencia_fechas_reporte_gastos

```

```
BEFORE INSERT OR UPDATE ON modulo5.reporte_gastos_acumulados
FOR EACH ROW
EXECUTE FUNCTION modulo5.fn_consistencia_fechas_reporte_gastos();
```

Nota sobre TGR-M05-03 y TGR-M05-15: Ambos se ejecutan en **BEFORE INSERT** sobre la misma tabla. En PostgreSQL los triggers se ejecutan en orden alfabético por nombre dentro del mismo evento, por lo que **trg_calculo_costo_total_consumo_alimento** se ejecuta antes que **trg_validacion_estado_tipo_alimento_activo**. Sin embargo, ambas funciones consultan el estado del tipo de alimento, por lo que en la implementación final del **.sql** puede ser conveniente fusionar ambas en una sola función para evitar la doble consulta, manteniendo la separación conceptual solo a nivel de documentación.